Food Truck Management System

A full-stack POS application for modern food truck operations
Designed to simulate real-world business workflow

Noemi Delgadillo Roldan
Per Scholas | Accenture / PeopleShores | April 2026

Problem & Solution

BEFORE

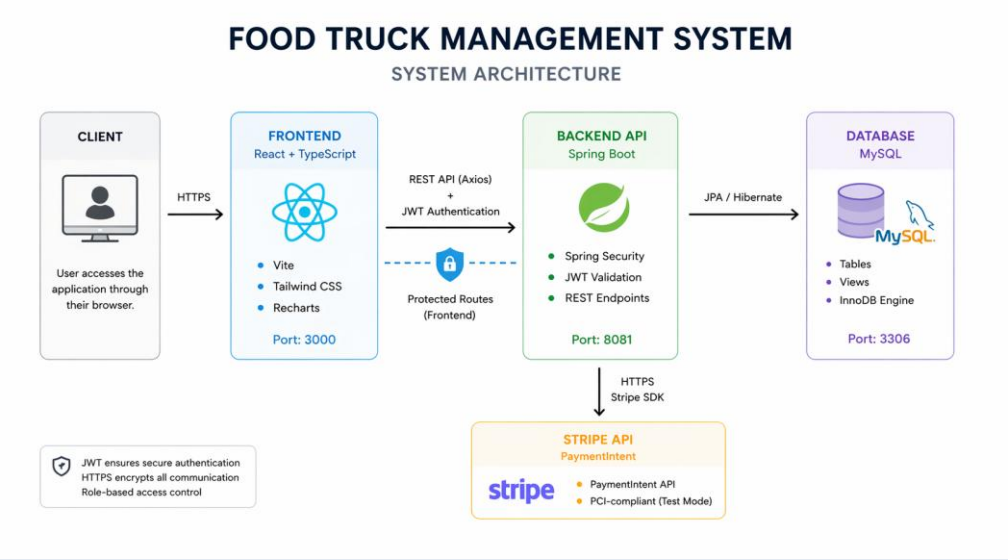
- Paper-based order management — errors during peak hours
- No real-time visibility on product availability
- Manual expense tracking — time-consuming and error-prone
- No sales data or business insights

->

AFTER

- Digital POS — orders created and tracked in real time
- Instant availability toggle for menu items
- Automated expense tracking with categories and reports
- Analytics dashboard with charts and business insights

System Architecture Overview



Frontend Layer

React 18 + TypeScript
Vite · Tailwind · Recharts

Backend Layer

Spring Boot 4
Spring Security · JWT

Database Layer

MySQL 8.0
6 tables + 3 views

Payments Layer

Stripe API
PaymentIntent flow
PCI-compliant payment handling
(test environment)

Live Demo

Application running locally
Full-stack integration (frontend, backend, database, payments)

Backend: Spring Boot | Port 8081 **Frontend:** React | Port 3000

Live Demo — Key Features

Dashboard

Real-time stats for admin
(orders, revenue, expenses)

New Order

Cart with notes per item
Cash & Stripe payment

Orders

Status tracking & updates
Kitchen modal for staff

Menu Management

Toggle availability
Product CRUD (admin)

Expenses

Track costs by category
(admin only)

Reports

Sales charts & trends
Popular items analysis

Technical Deep Dive

useReducer for Cart State

4 actions: ADD_ITEM, REMOVE_ITEM, UPDATE_QUANTITY, CLEAR_CART · All cart logic centralized in cartReducer.ts — predictable & testable · Notes per item tracked in cart state · No Redux needed — built-in React hook

MySQL Views for Reporting

3 database views: daily_sales, monthly_expenses, popular_items · Aggregation in DB — faster than computing in Java or JavaScript · Mapped as @Immutable JPA entities with composite keys (@IdClass) · Feeds Reports page charts (BarChart, PieChart, LineChart)

Stripe PaymentIntent Flow

Backend creates PaymentIntent — card data never touches our server · clientSecret returned to frontend securely · Stripe.js CardElement handles PCI-compliant card input · PCI-DSS compliant by design — test mode for demo

Security & Local Setup

JWT Authentication Flow

- 1. POST /api/auth/login with credentials
- 2. Backend validates & generates JWT token
- 3. JWT stored securely in localStorage (frontend)
- 4. Bearer token sent on every API request
- 5. Spring Security validates on each request
- 6. 401 Unauthorized → frontend redirects to login

Role-Based Access Control

Feature	ADMIN	EMPLOYEE
Products — CRUD	Yes	View only
Toggle Availability	Yes	Yes
Orders — Create & Update	Yes	Yes
Expenses & Reports	Yes	No
User Management	Yes	No

Lessons Learned

01

JPA Validation Errors

Transaction failures taught me to align frontend and backend validation constraints from the start — seed data must also respect all entity constraints.

02

Secrets in Source Control

Accidentally committing a Stripe secret key — GitHub's push protection blocked it. Learned to use environment variables and .gitignore from day one.

03

Backend First Development Approach

Building and testing the API before the frontend saved significant rework. TypeScript interfaces kept frontend-backend contracts aligned throughout.

?

Questions?

Thank you for your attention!

Noemi Delgadillo Roldan
Per Scholas | Accenture / PeopleShores | April 2026

Food Truck Management System